JavaScript (Parte 1 y 2)

Desarrollo de Aplicaciones en Entornos Web
Curso 2024/2025

Motivación

- Añadir capacidades de interacción a las páginas web.
- Ejemplo**: validación de datos en formularios**.
- **Estrategias**:
  - Compilación: Applets Java, ActiveX*,* **Adobe Flash**
  - Interpretación → **JavaScript**

- En general, la **programación dinámica en HTML** permite:
  - Controlar el contenido y apariencia de un documento en respuesta a los eventos de usuario.

Contenido

- Parte I:
  - **JavaScript** y la web
  - El Lenguaje JavaScript
- Parte II: **Expresiones Regulares** en JavaScript
- Parte III: El modelo de objetos **DOM**
- Parte IV: **Validación de Formularios**.

Parte I: JavaScript y la web

- Lenguaje **interpretado** incluido en documentos HTML.
  - Tiene un **propósito específico**: **actuar en páginas web**

- **¿Relación con Java?**:
  - En general, hay poco parecido entre los lenguajes.
  - En las construcciones imperativas es más parecido al lenguaje C.

- **Otros lenguajes similares** (*Microsoft*)**:** JScript y VBScript

- Estandarización: **ECMAScript**

JavaScript vs Java

JavaScript	Java
Orientado a objetos. Sin distinción entre tipos de objetos. La herencia se modela mediante el mecanismo de prototipado, y es posible añadir propiedades y métodos a cualquier objeto de forma dinámica.	Basado en clases. Los objetos se dividen en clases e instancias. Toda la herencia está sujeta a una jerarquía de clases. Las clases e instancias no pueden obtener nuevas propiedades o métodos de forma dinámica.
No se declaran los tipos de datos de las variables (tipado dinámico).	Los tipos de datos de las variables deben ser declarados (tipado estático).

Aplicaciones de JavaScript en la web

- Hacer que una página web reaccione a la **interacción del usuario**. Ejemplo: desplegar un menú al pulsar un enlace.

- **Preprocesar datos** en el cliente antes de que sean enviados al servidor (**validación de formularios**).

- **Cambiar el contenido y el estilo** dinámicamente en respuesta a la interacción del usuario (**programación dinámica en HTML**)

Usos no permitidos

- **Principio de diseño del lenguaje**:
  - Proteger la privacidad de los usuarios y evitar accesos no autorizados a sus ordenadores

- Por tanto, NO podemos:
  - Establecer o recuperar las preferencias del navegador, cambiar la apariencia de la ventana principal, accionar los botones o la impresión.
  - Ejecutar una aplicación.
  - Leer o escribir ficheros o directorios.
  - Abrir conexiones de red.
  - Enviar correos sin intervención del usuario.

Historia

- JavaScript creado en **1995** por *Brendan Eich*, ingeniero de Netscape
  - Publicado con Netscape 2 a principios de 1996.
- Inicialmente LiveScript, renombrado para aprovechar la popularidad del lenguaje Java
  - Ambos lenguajes tienen muy poco en común
- Netscape presentó el lenguaje al ECMA Internacional
  - Primera edición de la norma ECMAScript en 1997

- Actualización más importante: ECMAScript 3 en 1999
- La cuarta edición fue abandonada.
- ECMAScript 5 en 2009.
- **ECMAScript 6** en 2015
- ECMAScript 11 en 2020

https://es.wikipedia.org/wiki/ECMAScript#Versiones

Ejecutando JavaScript

- Probar JavaScript en entorno web.

- P.Ej: Firefox
  - Abrir la Consola Web, seleccionar "Consola web" desde el menú "Desarrollador web" que se encuentra en el menú "Herramientas" en Firefox
  - Aparece en la parte inferior de la ventana del navegador.
  - Usar línea de comando para ejecutar JavaScript, y el resultado aparece en el panel de arriba
  - Entrada Multi-línea: **Ctrl+B** (Windows/Linux) ó **Cmd+B** (macOS).

Hola Mundo

```
// Declaramos una funcion llamada saludar, que lleva
// ‘nombre' como argumento.

function saludar(nombre) {
    return "Hola " + nombre;
}

saludar("Usuario"); // Nos devolvera "Hola Usuario".
```

El Lenguaje JavaScript

- JavaScript **no** tiene un concepto de **entrada o salida**
  - **console.log()**
  - **console.error()**
- Diseñado para funcionar como lenguaje de script, en un entorno: navegador (V8 en Chrome, SpiderMonkey en Mozilla) ó *node.js* (V8).
  - Depende de los mecanismos de este entorno para la comunicación
- JavaScript es un lenguaje **dinámico** orientado a objetos.
- Tiene tipos y operadores, objetos básicos y métodos.
- JavaScript **no tiene clases**.
  - En cambio, la funcionalidad de clase se consigue mediante **prototipos** de objetos.

Características del Lenguaje

- **Distingue mayúsculas/minúsculas en los identificadores.**
- Comentarios como en Java y C (***//*** y ***/* … */***)
- **Declaración de variables** y **parámetros**:
  - **No se declaran tipos**
    - Ej.: `let i;`
  - Misma variable almacena varios tipos de datos
  - **Alcance** variables: global y local (bloque/función)

Características del Lenguaje

- Ignora espacio en blanco para formateo
  - Tabulaciones, nuevas líneas, etc.
- No es obligatorio terminar con punto y coma (**;**)

- **Operadores:** similares a C/Java.
- **Control de flujo** (condicionales, bucles)
  - similares a C/Java
  - **switch** admite cadenas

Características del Lenguaje

Syntactic Sugar: nuevos aspectos del lenguaje, cuya novedad reside en la sintaxis y no en nuevas funcionalidades.

- **for...in** itera sobre todos los elementos de un **objeto**, en un orden arbitrario.
  - Bucle iterará **solo** sobre las **propiedades definidas por el usuario**.
- **for...of** ejecuta un bloque de código para cada elemento de un objeto iterable, como lo son: String, Array, etc..
  - La sintaxis de **for...of** es específica para las **colecciones**
- Es posible usar **const** en lugar de **let** si no se va a modificar la variable dentro del bloque.

```
var obj = {a: 1, b: 2, c: 3};
for (const prop in obj) {
    console.log(`obj.${prop} = ${obj[prop]}`); //syntactic sugar
} // Salida: "obj.a = 1" "obj.b = 2" "obj.c = 3"

let iterable = [10, 20, 30];
for (let value of iterable) {
       value += 1;
       console.log(value);
} // 11 21 31

let iterable = "boo";
for (let value of iterable) {
    console.log(value);
} // b o o
```

Variables

- Nombre de las variables
  - El primer carácter debe ser una letra o un guión bajo (_) o un dólar ($).
  - El resto de caracteres pueden ser letras, números, guiones bajos (_) y símbolos de dólar ($).

Variables - var

- **Antes de ECMAScript 6,** Javascript no tenía ámbito de sentencias de bloque
- Si la variable **no** se declara mediante el operador var, automáticamente se crea una variable global con ese identificador y su valor

```
function calculo() {
    var variable1 = 16; //variable1 es LOCAL
    variable2 = variable1 + 4; //variable2 es GLOBAL}
var variable1 = 16; //variable1 es GLOBAL
function calculo() {
    variable2 = variable1 + 4; //variable2 es GLOBAL}
if (true) {
    var variable3 = 1; //variable3 es GLOBAL}
```

- Si colisión entre variable global y local, prevalece local

Variables

```
// mala práctica
variable1 = ‘javier’

// evitar con...
‘use strict’
variable1 = ‘javier’ // ERROR
```

Variables - let y const

- **ECMAScript 6**
- **let**
  - Declara una variable local en un **ámbito de bloque**, inicializándola opcionalmente a un valor.
  - Por ejemplo, `let y = 13`. Esta variable puede ser usada para declarar una variable local en un bloque de ámbito.
  - Una variable declarada usando la sentencia **var** o **let** sin asignarle un valor inicial tiene el valor **undefined**.
  - https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/let
  - https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/var
- **const**
  - Declara una constante de sólo lectura en un ámbito de bloque.
  - Inicializar al declarar

- Al intentar acceder a una variable no declarada dará como resultado la excepción ***ReferenceError***

```
var a;
console.log('El valor de a es ' + a);
                    // El valor de a es undefined

console.log('El valor de c es ' + c);
                    // Uncaught ReferenceError: c no está definida
let x;
console.log('El valor de x es ' + x);
                    // El valor de x es undefined
```

- Antes de ECMAScript6:

```
if (true) {
   var x = 5;
}
console.log(x); // x vale 5
```

- Declaración ***let*** introducida en ECMAScript6:

```
if (true) {
   let y = 5;
}
console.log(y);
// ReferenceError: y no está definida
```

Variable hoisting

- **Hoisting:** referenciar a una variable declarada con **var** más tarde, sin obtener una excepción.
  - Las variables en JavaScript son en cierto sentido "**elevadas**" a la parte superior de la función o declaración
  - valor *undefined*

```
console.log(x === undefined);   // true
var x = 3;
```

Ejemplo hoisting

```
var myvar = "my value";
(function() {
  console.log(myvar);    // undefined
  ???
})();
```

??? - ¿qué se debe poner para que la sentencia anterior muestre undefined?

Ejemplo hoisting

```
var myvar = "my value";
(function() {
  console.log(myvar);    // undefined
  var myvar = "local value";
})();
```

```
var x;
console.log(x === undefined); // true
x = 3
```

- Debido al **hoisting**, todas las declaraciones **var** en una función deben colocarse lo más cerca posible de la parte superior de la función.
  - Esta buena práctica mejora la **claridad** del código.

- En ES2015, **let** (y **const**) no elevan la variable a la parte superior del bloque: **NO hoisting**
  - Hacer referencia a la variable let/const en el bloque antes de la declaración de la variable → resulta en **ReferenceError**.

```
console.log(x); // ReferenceError
let x = 3;
```

Función hoisting

- Para las funciones, solo la **declaración** de la función se eleva a la parte superior y **no** la **expresión** de la función.

```
/* Declaración de la función */
foo();  // "bar"
function foo() { console.log("bar"); }

/* Expresión de la función */
baz(); // TypeError: baz no es una función
Var baz = function() { console.log("bar2"); };
```

Constantes

- Palabra clave ***const***
  - ***const*** PI = 3.14;

- Las reglas de ámbito para las constantes son las mismas que las de las variables **let** en un ámbito de bloque.

  - No se puede declarar una constante con el mismo nombre que una función o una variable en el mismo ámbito.

Tipos de Datos - Primitivos

- **Tipos Primitivos:** undefined, null, boolean, number (para enteros y reales) y string.
  - *undefined*: indefinido, variable sin inicializar
- El operador typeof para determinar el tipo de dato que almacena una variable.

```
variable1 = "hola mundo";
typeof variable1;  // devuelve "string”
let variable2;
typeof variable2;  // devuelve "undefined”
```

- Tipo null.
  - similar a *undefined* (undefined == null)
  - para representar objetos que en ese momento no existen

```
let nombreUsuario = null;
```

Tipos primitivos

- Tipo numérico
  - Si es decimal, utilizar el punto (.)

```
let variable1 = 10;
let variable2 = 3.14159265;
```

- Tres valores numéricos especiales:
  - Infinity y –Infinity : demasiado grandes y JavaScript no puede manejar
  - NaN : *Not a Number*

```
let variable1 = 3;
les variable2 = "hola";
isNaN(variable1);  // false
isNaN(variable2);  // true
```

Tipos primitivos

- Tipo boolean:
  - true, false → son valores especiales, no se corresponden con números ni con otro tipo.

```
var variable1 = true;
var variable2 = 1;
```

- Tipo cadena:
  - entre comillas simples o dobles

```
var variable1 = "hola";
var variable2 = 'mundo';
```

  - Cada carácter en una posición
    - primer carácter en posición 0.
  - *"mecanismo de escape": caracteres especiales empiezan con la barra \:* `\n \t \' \" \\`

Tipos de Datos - Conversión

- Conversión implícita de tipos o uso de toString:

```
var i = "3" + 3; // i == "33"
var variable1 = true;
variable1.toString();  // devuelve "true" como cadena de texto
```

- Conversión cadenas a números:

```
var i = parseInt("10");
var f = parseFloat("10.2");
```

- Conversión números a cadenas:

```
var s = "" + 10;
```

- **Igualdad** ==
  - Aplicada a objetos compara referencias.
  - Tipos primitivos: realiza conversión tipos: `3 == "3"; // true`
- **Identidad** ===
  - Sin conversión de tipos: `3 === "3"; // false`

Tipos de Datos - Otros

- Cualquier valor puede convertirse a booleano de acuerdo a las siguientes reglas:
  - **false**, **0**, la cadena vacía (**""**), **NaN**, **null**, y **undefined** todos son **false**
  - Todos los demás son **true**
- **Symbol** es un tipo de dato primitivo en cuyos casos son **únicos** e **inmutables**
  - Cada valor del tipo Symbol tiene asociado un **valor** del tipo **primitivo** que sirve únicamente como descripción del símbolo.
  - El objeto de Symbol es un **envoltorio** alrededor de un primitivo.

```
var sym2 = Symbol("foo");
Symbol("foo") === Symbol("foo");// false
var sym = new Symbol();        // TypeError
```

- Los Symbols son utilizados como **identificador** (valores únicos) de propiedades de objeto
  - Para crear y almacenar valores como integer y string que no van cambiar.

Tipos de Datos – Tipos de Referencia

- JavaScript **no** define el concepto de **clase/objeto**
- Tipos de referencia se asemejan a las **clases**
- Tienen propiedades y métodos
- Clases/Objetos del lenguaje:
  - ***B**oolean, **N**umber, **S**tring*
  - Arrays
  - Fechas (*Date*)
  - **Funciones**
- Primitivos se manejan por valor y objetos se manejan por referencia (= Java).

- Palabra reservada **new** y el nombre del tipo (clase/objeto)

```
var variable1 = new String("hola mundo");
```

- Una clase para cada tipo primitivo: *Boolean, Number* y *String*

```
var longitud = variable1.length;
longitud = "hola".length; //convierte string a String
```

Object

- La clase **Object**: base a partir de la cual heredan el resto de clases.
  - Constructor
  - Propiedad *prototype*

- *Conversión explícita entre tipos primitivos y sus correspondientes tipos de referencia:*

```
var numero = new Object(5);            // numero es de tipo Number
var cadena = new Object("hola mundo");// cadena es de tipo String
var conectado = new Object(false);    // conectado es de tipo Boolean
```

Number

- La clase **Number**:
  - Métodos útiles: valueOf y toFixed

```
var variable1 = new Number(16);
var variable2 = variable1.valueOf();  // variable2 = 16

var variable1 = new Number(3.141592);
var variable2 = variable1.toFixed();   // variable2 = 3
var variable3 = variable1.toFixed(2);  // variable3 = 3.14
var variable4 = variable1.toFixed(10); // variable4 = 3.1415920000
```

String

- **Concatenación**: operador **+**
- **Tamaño:** propiedad **`length`**
- Acceso a un carácter: método **`charAt`**`(i)`
- Buscar **subcadenas**: método **`indexOf`**(“subcadena”), **`lastIndexOf`**(“subcadena”)
- Extraer subcadenas: método **`substring`**`(i, j)`
- Pasar a **may/min**: **`toUpperCase`**(s), **`toLowerCase`**(s)
- Obtener subcadenas de acuerdo a un separador: **`split`**(“separador”)

```
var mystring = "Hello, World!";
mystring[0] = 'L'  // No produce ningún efecto
mystring[0];       // Esto devuelve "H"
```

- **Expresiones incrustadas** (cadenas delimitas con acento francés ` ${} `)
  - *plantillas de cadenas de texto* **(syntactic sugar)**

```
var a = 5;
var b = 10;
console.log("Quince es " + (a + b));
console.log(`Quince es ${a + b}`);
```

Arrays

- Creación:

```
var array = new Array(); var array1 = [“juan”, 5, false];
var array2 = new Array(10); var array3 = new Array(2, ”abc”);
```

- Acceso a un array:
  - Índice a partir de 0. **Longitud**: propiedad **`length`**
  - **Indexación con cadenas**: `array[“cantidad”] = 3;`
    - *Arrays* ***asociativos***
- **Heterogéneos y dispersos**:

```
array[0] = 20;
array[20] = “hola”;
```

→ `array[5] == undefined, length == 21`

- El array aumenta de forma dinámica al añadir elementos

Arrays

- Manejo del array como una pila:
  - **`pop`**() : elimina **último** elemento del array
  - **`push`**(ele) : añade *ele* como **último** elemento del array

  - **`unshift`**() : elimina **primer** elemento del array
  - **`shift`**(ele) : añade *ele* como **primer** elemento array

- Otras funciones:
  - **`reverse`**() : invierte el orden del contenido del array

Arrays asociativos

- Los *objetos* son arrays asociativos que almacenan sus propiedades y métodos:

```
elObjeto.id = "10";
elObjeto.nombre = "Objeto de prueba";
elObjeto.muestraId = function() {
            alert("El ID del objeto es " + this.id);
            }
```

  - Uso de **this** para referirse al objeto invocado
  - No es necesario que la clase tenga definida previamente la propiedad.

```
elObjeto['id'] = "10";
elObjeto['nombre'] = "Objeto de prueba";
```

- Relación de clientela entre objetos:

```
var Aplicacion = new Object();
Aplicacion.Modulos = new Array();
Aplicacion.Modulos[0] = new Object();
Aplicacion.Modulos[0].titulo = "Lector RSS";

var inicial = new Object();
inicial.estado = 1;

Aplicacion.Modulos[0].objetoInicial = inicial;
```

Fechas

- Clase **Date**: representa una fecha y hora

- **Construcción:**
  - **Fecha actual**: `var hoy = new Date();`
  - Los meses mediante valor numérico: empieza en 0 (*Enero*) y termina en 11 (*Diciembre*)

```
var fecha = new Date(2004, 9, 20);
                    // año, mes y día → 20/10/2004

var fecha = new Date(2004,9, 20, 22, 50, 50);
                                  // 20/10/2004, 22:50:50
```

Fechas

- **Acceso y modificación**:
  - Se definen métodos **get/set** para acceder a sus propiedades.

getTime() : milisegundos

getFullYear() : 4 digits; getYear() : 2 dígitos

getDate() : devuelve el número del día del mes

getDay(): número del día de la semana (0 : Domingo, 6: Sábado)

getHours(), getMinutes(), getSeconds(), getMilliseconds()

- **Aritmética** de fechas:
  - Ejemplo: podemos incrementar el día del mes en 45 y actualiza el mes y el día para dentro de 45 días:
  - `fecha.setDate(fecha.getDate() + 45);`

Internacionalización

- El objeto **Intl** es un espacio de nombres para el API de Internacionalización
  - Provee comparación de cadenas sensibles al idioma, formato de números, y formato de fechas con horas.
  - El constructor para los objetos **Collator**, **NumberFormat**, y **DateTimeFormat** son propiedades del objeto **Intl**.
    - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Collator

DateTimeFormat

```
var msPerDay = 24 * 60 * 60 * 1000;
var july172014 =
          new Date(msPerDay * (44 * 365 + 11 + 197));
var options = {  year: "2-digit", month: "2-digit",
                 day: "2-digit", hour: "2-digit",
                 minute: "2-digit", timeZoneName:
   "short" };

var americanDateTime =
          new Intl.DateTimeFormat("en-US",
   options).format;

console.log(americanDateTime(july172014));
          // 07/16/14, 5:00 PM PDT
```

Función eval()

- La función **eval(s)** evalúa un código JavaScript representado como una cadena de caracteres **s** (string)

```
eval("2 + 2");                  // devuelve 4
eval(new String("2 + 2"));      // devuelve un objeto
                                String que contiene "2 + 2"
```

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/eval

- Si utiliza *indirectamente*, invocándola a través de una referencia distinta de eval, a partir de ECMAScript 5, ésta funciona en el ámbito global, en lugar de hacerlo en el ámbito local

```
function test() {
    var x = 2, y = 4;
    console.log(eval("x + y”));
    // Llamada directa, utiliza el ámbito local, el resultado es 6

    var geval = eval;
    console.log(geval("x + y"));
    // Llamada indirecta, utiliza el ámbito global, a través de
    ReferenceError por que `x` es indefinida
}
```

Funciones

- **Function** raramente se utiliza de forma explícita

```
var miFuncion = new Function("a", "b", "return a+b;");
```

  - último argumento de la llamada se considera como las instrucciones

- **Declaración**:

```
function suma (a, b) {
    return a + b; }
```

- No se declaran los tipos de los parámetros ni el valor de retorno.
  - Paso de parámetros por valor.

Funciones

- El nombre de una función (ejemplo “suma”) es un objeto que puede ser asignado a una variable.

```
var resultado = suma(2, 3);
```

- No hay comprobación de tipos: no se aseguran los tipos de los parámetros en la llamada.

- Funciones anónimas:

```
var miFuncion = function(a, b) { return a+b; }
```

  - Para cuando se necesita definir funciones sencillas que sólamente se utilizan una vez.

Funciones

- Si se pasan **más** parámetros los definidos, los parámetros sobrantes se ignoran

- Si se pasan **menos** parámetros los definidos, se asigna el valor *undefined*.

  - *arguments* : array especial con los parámetros de invocación de la función
  - Manejo de argumentos variables

Funciones - arguments

```
arguments.length        // numero de argumentos invocados

arguments[0]            // primer argumento

arguments.callee        // funcion invocada

arguments.callee.length// numero de parametros definidos
                          en la funcion invocada
```

Recursividad

- Una función puede referirse y llamarse a sí misma. Hay tres formas de una función para referirse a sí misma:
  - El nombre de la función
  - arguments.callee
  - Una variable en el ámbito en el que se refiere a la función

```
var foo = function bar() {
    // las sentencias van aqui
};
```

  - `bar()`
  - `arguments.callee()`
  - `foo()`

Funciones – call() , apply()

- ***apply()*** y ***call()*** permiten ejecutar una función como si fuera un método de otro objeto
- ***call***():

```
function miFuncion(x) { return this.numero + x; }
var elObjeto = new Object();
elObjeto.numero = 5;
var resultado = miFuncion.call(elObjeto, 4); // 9
```

  - primer parámetro es objeto receptor (this) resto parámetros son los parámetros de la función
- ***apply***() idéntico call():
  - los parámetros se pasan como un array.

```
var resultado = miFuncion.apply(elObjeto, [4]);
```

Encadenamiento de Ámbito

- Se puede anidar una función dentro de una función.
  - La función anidada (interna) es privada a su función contenedora (externa).

- También forma *un* ***cierre***.
  - Un cierre, es una expresión (típicamente una función) que puede tener variables libres junto con un entorno que ata esas variables (que "cierra" la expresión).

- Dado que una **función anidada es un cierre**, esto significa que una función anidada puede "***heredar***" los **argumentos y variables** de su función **contenedora**.
  - En otras palabras, la función interna contiene el ámbito de la función externa.

- Los cierres pueden contener múltiples ámbitos; que contienen de forma recursiva el ámbito de las funciones que los contienen

- En resumen:
  - La **función interna sólo se puede acceder** a partir de sentencias dentro de la función **externa**.
  - **La función interna forma un cierre**:
    - la función interna **puede** utilizar los argumentos y variables de la función externa
    - mientras que la función externa **no puede** utilizar los argumentos y las variables de la función interna.

```
function A(x) {
   function B(y) {
      function C(z) {
         console.log(x + y + z);
      }
      C(3);
   }
   B(2);
} A(1); // registra 6 (1 + 2 + 3)
```

Funciones Constructoras

- JavaScript no existe el concepto de constructor
- JavaScript *emula* el funcionamiento de los constructores mediante el uso de funciones

```
var elArray = new Array(5);
```

  - Array es el nombre de una función que se ejecuta para crear el nuevo objeto

```
function Factura(idFactura, idCliente){
        this.idFactura = idFactura;
        this.idCliente = idCliente;
}
```

  - La función constructora inicializa las propiedades de cada objeto mediante el uso de la palabra reservada **this**.

```
var laFactura = new Factura(3, 7);
```

Prototype

```
function Factura(idFactura, idCliente) {
  this.idFactura = idFactura;
  this.idCliente = idCliente;
  this.muestraCliente = function() { alert(this.idCliente);  }
  this.muestraId = function() { alert(this.idFactura);  }
}

var laFactura = new Factura(3, 7);
laFactura.muestraCliente();
var otraFactura = new Factura(5, 4);
```

- ***Inconveniente**: las funciones muestraCliente*() y *muestraId*() se crean de nuevo por cada objeto creado !!
- Penalización en el rendimiento y el consumo excesivo de recursos

Prototype

- Todos los objetos incluyen una referencia interna a otro objeto llamado ***prototype***.
  - Cualquier propiedad o método que contenga el objeto prototipo está presente de forma automática en el objeto original (común a todos los objetos, semántica ***static***).

- Todo objeto "hereda" de todas las propiedades y métodos del objeto prototype.
  - Cada tipo de objeto diferente hereda de un objeto *prototype* diferente.

Prototype

```
function Factura(idFactura, idCliente) {
    this.idFactura = idFactura;
    this.idCliente = idCliente;
}
Factura.prototype.muestraCliente = function() {
        alert(this.idCliente);
    }
Factura.prototype.muestraId = function() {
        alert(this.idFactura);
    }
```

- ***Inconveniente***: la propiedad *prototype* permite reescribir propiedades y métodos!!

Prototype

- Modificar y añadir propiedades y métodos de los *objetos predefinidos* por JavaScript

```
Array.prototype.indexOf = function(objeto) {
   //…nuevo comportamiento…
}
```

- *Cualquier* clase nativa de JavaScript puede ser modificada mediante la propiedad *prototype*

```
String.prototype.trim = function() {
   return this.replace(/^\s*|\s*$/g, '');
}
```

Otros: typeof vs instanceof

- Operador typeof devuelve el valor Object para cualquier (tipos de referencia).

- Operador **instanceof**:

```
var variable1 = new String("hola mundo");
typeof variable1;              // devuelve "object”
variable1 instanceof String;   // devuelve true
```

Otros: Excepciones

```
try {
   // .. operaciones ..
} catch(excepcion) {   // se ejecuta si una excepcion
   alert(excepcion);
} finally {            // se ejecuta siempre
   alert(”bloque finally");
}
```

- No se distinguen tipos de excepciones
- El bloque *catch* permite indicar el nombre del parámetro para una excepción.
  - Este identificador sólo está definido dentro del bloque *catch*
  - Útil para obtener información sobre la excepción.

Excepciones

- Se pueden lanzar exceptions utilizando *throw* y manejarlas usando sentencias *try...catch*

- Casi cualquier objeto puede ser lanzado en JavaScript.
  - Sin embargo, no todos los objetos lanzados son iguales.
  - Es común arrojar números o cadenas como errores
  - Es más efectivo usar uno de los tipos de excepción creados específicamente

```
throw 'Error2';   // String type
throw 42;         // Number type
throw true;       // Boolean type

function UserException(message) {
  this.message = message;
  this.name = 'UserException';
}
UserException.prototype.toString = function() {
  return `${this.name}: "${this.message}"`;
}
throw new UserException('Value too high');
```

```
try {
  throw 'myException';    // generates an exception
}
catch (err) {
  logMyErrors(err);       // pass exception object to error
    handler
}
finally {
  …                       // always executed
}
```

Otros: Errores

- Se lanzan objetos de error cuando se producen errores de tiempo de ejecución.
- El objeto *Error* también se puede usar como un objeto base para excepciones definidas por el usuario.
- Los errores en tiempo de ejecución provocan la creación y el lanzamiento de nuevos objetos de *Error*.

- Además del constructor genérico de errores, hay otros siete constructores de errores core en JavaScript:
  - **EvalError**: al usar funcion *eval()*
  - **InternalError**: error interno en el motor de JavaScript
  - **RangeError**: error numérico, fuera de rango
  - **ReferenceError**: de referenciar una referencia ya inválida
  - **SyntaxError**: error de sintáxis al parsear código en *eval()*
  - **TypeError**: variable o parámetro no es de tipo válido
  - **URIError**: parámetros no válidos a *encodeURI()* o *decodeURI()*

- Propiedades de Error:
  - *name* y *message*

```
try {
  throw new Error('Whoops!')
} catch (e) {
  console.error(e.name + ': ' + e.message)
}
```

- Capturar errores específicos:

```
try {
  foo.bar()
} catch (e) {
  if (e instanceof EvalError) {
    console.error(e.name + ': ' + e.message)
  } else if (e instanceof RangeError) {
    console.error(e.name + ': ' + e.message)
  }
  // ... etc
}
```

```
function check(n)
{    if( !(n >= -500 && n <= 500) )
    { throw new RangeError("The argument must be between -500 and 500.") }
}
try {
     check(2000) }
catch(error) {
     if (error instanceof RangeError)  { ...

function check(value)
{    if(["apple", "banana", "carrot"].includes(value) === false)
    { throw new RangeError('The argument must be an "apple", "banana", or
     "carrot".') }
}
try {
     check("cabbage”) }
catch(error) {
     if (error instanceof RangeError)  { ...
```

Closures

- Las **clausuras** (informalmente llamadas cierres) son una de las características más poderosas de JavaScript.

- Permite debido al anidamiento de funciones y concesiones a la **función interna**, el **acceso total** a todas las variables y funciones definidas dentro de la **función externa** (y a todas las variables y funciones a las cuales la función externa tiene acceso).

- Sin embargo, la **función externa no tiene acceso** a las variables y funciones definidas dentro de la función **interna**.
  - Esto provee una especie de **seguridad** a las variables de la función interna.
- Ya que la función interna tiene acceso al ámbito de la función externa, las **variables** y funciones definidas **en la función externa vivirán más tiempo que la función externa en sí misma** si la función interna las administra para sobrevivir más allá del ciclo de vida de la función externa.

- Una **clausura es creada** cuando la función interna es, de alguna manera, **hecha disponible** a cualquier ámbito **fuera** de la función **externa**.

```
var pet = function(name) {
    var getName = function() {
            return name;
            // La funcion interna tiene aceso a la variable "name"
    de la funcion externa
    }
    return getName;
    // Devolver la funcion interna, por lo tanto exponiendola a un ambito
    exterior
}
myPet = pet("Vivie");
myPet(); // Devuelve "Vivie"
```

- Las variables **internas** de las funciones internas actúan como **almacenamientos seguros** para las variables y argumentos **externos**.
  - Mantienen "persistente", y aún segura, la información con la que necesitan trabajar las funciones internas.

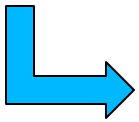
Ejemplo de motivación

```
// Initiate counter
var counter = 0;

// Function to increment counter
function add() {
  counter += 1;
}

// Call add() 3 times
add();
add();
add();

// The counter should now be 3
```



```
// Initiate counter
var counter = 0;

// Function to increment counter
function add() {
  var counter = 0;
  counter += 1;
}

// Call add() 3 times
add();
add();
add();

//The counter should now be 3. But it is 0
```

```
// Function to increment counter
function add() {
  var counter = 0;
  counter += 1;
  return counter;
}

// Call add() 3 times
add();
add();
add();

//The counter should now be 3. But it is 1.
```

- **counter** = 0 ejecutado mas de una vez
- es necesario acceder a la función **plus**() desde fuera

```
function add() {
  var counter = 0;
  function plus() {counter += 1;}
  plus();
  return counter;
}
```

```
var add = (function () {
  var counter = 0;
  return function () {counter += 1; return counter}
})();

add();
add();
add();

// the counter is now 3
```

- A la variable **add** se le asigna el valor de retorno de una función de invocación automática.
- La función de invocación automática **solo se ejecuta una vez**.
- Establece el **contador** en **cero** (0) y devuelve una expresión de función.
- De esta forma, **add** se convierte en una función. La parte "maravillosa" es que **puede acceder al contador** en el ámbito principal.
- Esto se llama cierre (closure) de JavaScript.
- Permite que una función tenga variables **"privadas"**.
- El contador está protegido por el alcance de la función anónima y solo se puede cambiar con la función de agregar.

- Un **cierre (closure)** es una **función que tiene acceso al ámbito principal**, incluso **después** de que la función principal se haya **cerrado**.

Funciones flecha

- Las funciones flecha son siempre funciones anónimas.
- Dos factores influenciaron la introducción de las funciones flecha: funciones más cortas y léxico **this**.
  - ***Syntactic Sugar***

```
var a = [ "Hydrogen", "Helium", "Lithium", "
   Beryllium" ];
var a2 = a.map(function(s){ return s.length });
var a3 = a.map( s => s.length );
```

- Las funciones flecha no tienen **this** propio
  - Hasta antes de las funciones flecha, cada nueva función definía su propio valor **this** : un nuevo object en el caso de un constructor
- Las funciones flecha capturan el valor de this del **contexto circundante**

```
function Person(){
    this.age = 0;
    setInterval(() => {
                    this.age++;
// |this| se refiere al objeto instancia de Person.
                        }, 1000);
}
var p = new Person();
```

Parámetros rest

- La syntaxis de parámetros **rest** (en inglés) nos permite respresentar un número **indefinido** de argumentos en forma de **array**.

```
function multiply(multiplier, ...theArgs){
      return theArgs.map(x => multiplier * x);
   }
var arr = multiply(2, 1, 2, 3);
console.log(arr); // [2, 4, 6]
```

Modelo de Objetos en JavaScript

- JavaScript es un lenguaje orientado a objetos basado en prototipos en lugar de clases.

- Debido a esta diferencia, puede ser menos evidente cómo JavaScript te permite crear **jerarquías** de objetos y **herencia** de propiedades y sus valores

- Un lenguaje basado en prototipos, como JavaScript, **no** hace esta distinción entre **clase/ instancia**: simplemente tiene **objetos**

- Un lenguaje basado en prototipos toma el concepto de *objeto* ***prototípico****:*
  - un **objeto** que se utiliza como una **plantilla** a partir de la cual se obtiene el conjunto inicial de propiedades de un nuevo objeto.
  - cualquier objeto puede **especificar** sus propias **propiedades**, ya sea cuando es creado o en tiempo de ejecución.
- Adicionalmente, cualquier objeto puede ser utilizado como el ***prototipo*** **de otro objeto**
  - permitiendo al segundo objeto compartir las propiedades del primero **¿herencia?**.

- Las Clases en JavaScript, introducidas en ECMAScript 2015, son básicamente un retoque **sintáctico** sobre la herencia basada en prototipos de JavaScript.
  - ***Syntactic Sugar***
- La sintaxis ***class*** **no** introduce un nuevo modelo de herencia orientado a objetos en Javascript.

Employee

```
function Employee(){
  this.name = "";
  this.dept = "general";
}
```

Manager

```
function Manager(){
  this.reports = [];
}
Manager.prototype = new Employee;
```

WorkerBee

```
function WorkerBee(){
  this.projects = [];
}
WorkerBee.prototype = new Employee;
```

SalesPerson

```
function SalesPerson(){
  this.dept = "sales";
  this.quota = 100;
}
SalesPerson.prototype = new WorkerBee;
```

Engineer

```
function Engineer(){
  this.dept = "engineering";
  this.machine = "";
}
Engineer.prototype = new WorkerBee;
```

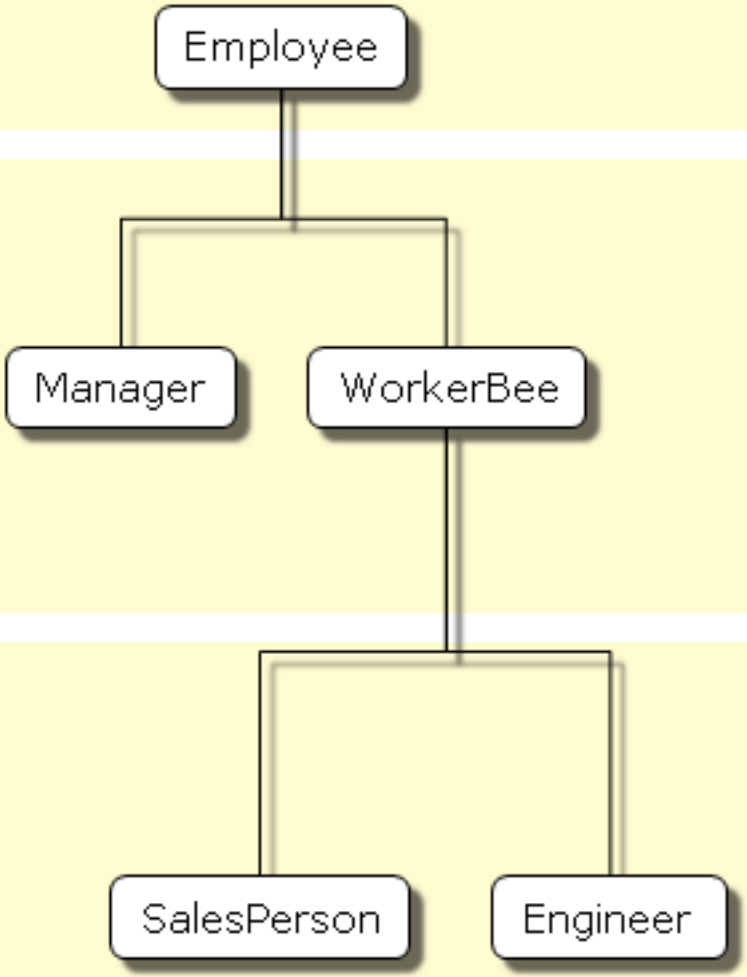
```
function Employee () {
  this.name = "";
  this.dept = "general";
}
```

```
function Manager () {
  this.reports = [];
}
Manager.prototype = new Employee;

function WorkerBee () {
  this.projects = [];
}
WorkerBee.prototype = new Employee;
```

```
function SalesPerson () {
   this.dept = "sales";
   this.quota = 100;
}
SalesPerson.prototype = new WorkerBee;

function Engineer () {
   this.dept = "engineering";
   this.machine = "";
}
Engineer.prototype = new WorkerBee;
```




```
var jim = new Employee;
// jim.name is ''
// jim.dept is 'general'
```

```
var sally = new Manager;
// sally.name is ''
// sally.dept is 'general'
// sally.reports is []

var mark = new WorkerBee;
// mark.name is ''
// mark.dept is 'general'
// mark.projects is []
```

```
var fred = new SalesPerson;
// fred.name is ''
// fred.dept is 'sales'
// fred.projects is []
// fred.quota is 100

var jane = new Engineer;
// jane.name is ''
// jane.dept is 'engineering'
// jane.projects is []
// jane.machine is ''
```

```
var mark = new WorkerBee;
```

- crea un nuevo objeto genérico
- establece implícitamente el valor de la propiedad interna [[**Prototype**]] (**__proto__**) con el valor de WorkerBee.prototype (objeto Employee)
- pasa este nuevo objeto como **this** a la función constructora de WorkerBee.

```
mark.name = "";
mark.dept = "general";
mark.projects = [];
```

- El objeto **mark** hereda valores para las propiedades **name** y **dept** de su objeto prototipico que enlaza en **mark.__proto__**.
- Se le asigna un valor local a la propiedad **projects** a través del **constructor** WorkerBee.
- De esta forma se heredan propiedades y sus valores

- Si añades una nueva propiedad a un objeto que se esta utilizando como el prototipo de una función *constructor*, dicha propiedad se añade a todos los objetos que heredan propiedades de dicho prototipo.

```
Employee.prototype.specialty = "none";
```

- Entonces el objeto mark también tienen la propiedad specialty con el valor "none".

- Definiciones JavaScript realizan un uso idiomático especial para asignar **valores por defecto:**

```
function Employee (name, dept) {
  this.name = name || "";
  this.dept = dept || "general";
}
```

- El operador lógico OR de JavaScript (||) evalúa su primer argumento.
- Si dicho argumento se convierte a true, el operador lo devuelve. Si no, el operador devuelve el valor del segundo argumento.

- La búsqueda de propiedades en la cadena de prototipos comienza en las propiedades locales del objeto y si no se encuentran localmente, se busca a través de la propiedad **__proto__** del objeto.

- La búsqueda continúa recursivamente, conociéndose como "**búsqueda en la cadena de prototipos**".

- La propiedad especial **__proto__** de un objeto recibe su valor en el momento en el que es creado
  - se le asigna el **valor de la propiedad prototype** de la función ***constructor*** usada para crear el objeto

- Así, la expresión `new Foo()` crea un objeto con `__proto__ == Foo.prototype`
- Los cambios que se realicen en las propiedades de `Foo.prototype` alteraran la búsqueda de propiedades de todos los objetos que se crearon mediante `new Foo()`.

- Todo objeto tiene una propiedad **__proto__** (salvo Object)
- Toda función tiene una propiedad **prototype**
- Es así como los objetos pueden relacionarse mediante '**herencia de prototipos'** con otros objetos.

```
var chris = new Engineer("Pigman, Chris", ["jsd"], "fiji");

chris.__proto__ == Engineer.prototype;
chris.__proto__.__proto__ == WorkerBee.prototype;
chris.__proto__.__proto__.__proto__ == Employee.prototype;
chris.__proto__.__proto__.__proto__.__proto__ == Object.prototype;
chris.__proto__.__proto__.__proto__.__proto__.__proto__ == null;


instanceOf (chris, Engineer)
instanceOf (chris, WorkerBee)
instanceOf (chris, Employee)
instanceOf (chris, Object)
```

- Las **clases** de javascript, introducidas en ECMAScript 2015, son una **mejora sintáctica** sobre la herencia basada en prototipos de JavaScript.
  - La sintaxis de las clases **no** introduce un nuevo modelo de herencia
  - Proveen una sintaxis mucho más clara y simple para crear objetos y lidiar con la herencia.

Declaración de clases

```
class Rectangle {
   constructor(height, width) {
      this.height = height; this.width = width;
   }
}
```

- Ojo! **declaraciones de clases** no son alojadas

```
const p = new Rectangle(); // ReferenceError
Class Rectangle {}
```

Expresiones de clases

- Una **expresión de clase** es otra manera de definir una clase.
- Las expresiones de clase pueden ser nombradas o anónimas.
- El nombre dado a la **expresión de clase** nombrada es local dentro del cuerpo de la misma.

```
// Anonima
var Poligono = class {
    constructor(alto, ancho) {
        this.alto = alto;
        this.ancho = ancho;
    }
};
// Nombrada
var Poligono = class Poligono {
    constructor(alto, ancho) {
        this.alto = alto;
        this.ancho = ancho;
    }
};
```

Constructor

- El método constructor es un método especial para crear e inicializar un objeto creado con una clase.

- Solo puede haber un método especial con el nombre "constructor" en una clase.
  - Si esta contiene mas de una ocurrencia del método **constructor,** se arrojará un *Error* SyntaxError

- Un **constructor** puede usar la *palabra reservada* **super** para llamar al **constructor** de una *superclase*

Métodos estáticos y super

- La *palabra clave* **static** define un método estático para una clase.
- Los métodos estáticos son llamados sin instanciar su clase y **no** pueden ser llamados mediante una instancia de clase.
- Los métodos estáticos son a menudo usados para crear funciones de utilidad para una aplicación.
- La palabra clave **super** es usada para llamar funciones del objeto padre.

Subclases: extend

```
class Animal {
    constructor(nombre) {
        this.nombre = nombre;
    }
    hablar() {
        console.log(this.nombre + ' hace un ruido.');
    }
}
class Perro extends Animal {
    hablar() {
        super.hablar();
        console.log(this.nombre + ' ladra.');
    }
}
```

Array y programación funcional

- **forEach:**

```
array1.forEach(element => console.log(element));
```

- **filter:**

```
const result = words.filter(word => word.length > 6);
```

- **find:**

```
const found = array1.find(element => element > 10);
```

- **map y flatMap:**

```
var arr1 = [1, 2, 3, 4];
arr1.map(x => [x * 2]);  // [[2], [4], [6], [8]]
arr1.flatMap(x => [x * 2]);  // [2, 4, 6, 8]
```

Programación Asíncrona - Promesa

- A partir de ECMAScript 6, Javascript con soporte para los objetos ***Promise***

- Una **promesa** es un concepto asociado a la programación que tiene como propósito evitar el encadenamiento de funciones callback,
  - favoreciendo, por tanto, la legibilidad del código.

- Formalmente, una promesa es un **objeto que es retornado inmediatamente tras una llamada asíncrona** a una función y que representa el posible valor (promesa) que se obtendrá de la llamada.

- El beneficio que aporta este objeto es que se puede emplear el valor que se espera obtener de la función, aunque este sea obtenido en el futuro.

- De este modo, es posible programar código secuencial en el que intervengan llamadas asíncronas a funciones.

- Estados de un objeto *Promise*:
  - *pending*: Estado inicial, no terminada exitosamente o rechazada.
  - *fulfilled*: operación exitosa.
  - *rejected*: operación fallida.
  - *settled*: Promise ha sido exitosa o fallida, pero no está pendiente.

Ejemplo Promise XMLHttpRequest es cargar una imagen

```
function imgLoad(url) {
     return new Promise(
                    function(resolve, reject) {
                         var request = new XMLHttpRequest();
                         request.open('GET', url);
                         request.responseType = 'blob';
                         request.onload = function() {
                              if (request.status === 200 {
                                   resolve(request.response);
                              } else {
                                   reject(Error('Image didn\'t load successfully;
                                   error code:' + request.statusText));}
                         };
                         request.onerror = function() {
                              reject(Error('There was a network error.'));
                         };
                         request.send(); });
}
```

async/wait

- El modificador **async** aplicado a una función se emplea para indicar que algunas de las instrucciones que contiene se ejecutarán de forma asíncrona.
-
- En el cuerpo de la función se emplea la palabra reservada **await** en aquellos cálculos que impliquen una llamada asíncrona que retorne una promesa.

- El propósito de las funciones async/await es que el uso de promesas sea **transparente** al programador.

- A su vez, la llamada a un función async retorna una promesa. El resultado es un modelo de **programación asíncrono que se expresado de forma secuencial.**

async/wait

- async: una función que siempre devuelve un *Promise*

```
async function f() {
   return 1; //Lo mismo que Promise.resolve(1);
}
f().then(alert); // 1
```

- **wait**: espera hasta que se obtiene Promise y lo devuelve

```
// works only inside async functions

let value = await promise;
```

```
async function f() {
  let promise = new Promise(
    (resolve, reject) =>
      { setTimeout(() =>
        resolve("done!"), 1000) });
  let result = await promise;
  // wait until the promise resolves (*)
  alert(result); // "done!"
}
f();
```

Situación del código JavaScript

- **Declaración en una página HTML**:

```
<script type="text/javascript" language="javascript">
…
</script>
```

  - Habitualmente el código se sitúa en la **cabecera.**

- **En un fichero externo** (útil para librerías de funciones):

```
<script type="text/javascript"
     src="libreria.js"></script>
```

- **En un manejador de evento.** Se estudiará más adelante.

Flujo de ejecución en JavaScript

- Al cargar la página que contiene/referencia código JavaScript:
  - Todas las sentencias se van ejecutando conforme se carga la página.
  - La declaración de funciones NO ejecuta la función, si no que solo carga la función en memoria.
  - El uso de código JavaScript en propiedades/eventos de elementos XHTML no genera su invocación.
    - La invocación se desencadena al producirse el evento asociado.

Parte II: Expresiones Regulares

- **Motivación**:
  - Búsqueda y reemplazo de un patrón en una cadena.
  - **Muy útiles para validar campos de un formulario**
- Patrones expresados de forma flexible y concisa.
- Ejemplos:
  - `var er = /hola/; // patrón "hola"`
  - `var er = /\d{3}/; // Número de 3 cifras`

- **Tipos de patrones**:
  - **Básico**: `/Hola/` → patrón "Hola"
  - **Insensible mayúsculas/minúsculas**: `/hola/i`

Caracteres especiales

- **\b** límite palabra
- **\d** dígito
- **\s** espacio en blanco
- **\w** letra, dígito o "_"
- **.** cualquier carácter excepto salto de línea
- **[ ]** conjunto de caracteres
- **[^]** cualquier carácter excepto …
- **[a-m]** rango de caracteres
- ***** 0 ó más apariciones
- **?** opcional
- **+** 1 ó más apariciones
- **{n}** n apariciones
- **{n, m}** entre n y m
- **^** comienzo línea
- **$** fin línea
- **( )** agrupación
- **|** alternativa
- **** carácter escape

Ejemplos

- **Cualquier dígito**: **\d**
  - `var er = /\d\d/;` → patrón de búsqueda formado por dos dígitos: 01, 12, etc.
- **Espacio en blanco**: **\s**
  - `var er = /hola\smundo/;` → secuencia “hola” seguida de un espacio y la secuencia “mundo”.
- **Carácter alfanumérico**: **\w** → letra, dígito o “_”
  - `var er = /\w\w\w/;` → tres caracteres alfanuméricos: “h32” “aaa”, “321”, “a_b”, etc.
  - No se incluyen la ñ y los caracteres acentuados en español.

Ejemplos

- **Cualquier carácter**: **.**
  - `var er = /abc.123/;` → secuencia “abc” seguida de cualquier carácter y finalizada con la secuencia “123”.
- **Conjunto de caracteres**: **[ ]**
  - `var er = /[abc]/;` → un carácter que puede ser ‘a’, ‘b’ o ‘c’.
- **Negación de un conjunto**: **[^]**
  - `var er = /[^abc]/;` → un carácter que no sea ni ‘a’, ‘b’ o ‘c’.
- **Conjunto expresado como rango ASCII**:
  - `var er = /[a-z]/;` → un carácter que va desde la ‘a’ hasta la ‘z’ en el código ASCII.
  - Nota: la ñ no se encuentra en el rango.

Ejemplos

- **Cualquier número de veces**: *****
  - `var er = /hola\d*/;` → secuencia ‘hola’ que va seguida de ningún carácter o un dígito que puede repetirse varias veces.
  - `var er = /a[bc]*/;` → patrón de búsqueda que empieza por ‘a’ y va seguido de ningún carácter o ‘b’ o ‘c’ repetidos varias veces: ‘abbbcb’, ‘accbcc’, etc.
- **Repetición al menos una vez**: **+**
  - `var er = /\d+/;` → un dígito que puede repetirse varias veces.
- **Opcional: ?**
  - `var er = /a?\d+/;` → una secuencia de dígitos que puede comenzar por ‘a’.
- **Número fijo de repeticiones**: **{n}**
  - `var er = /\d{3}/;` → números de tres cifras.

Ejemplos

- **Rango de repeticiones**: **{n, m}**
  - `var er = /\d{3, 5}/;` → números entre 3 y 5 cifras.
  - `var er = /[ab]{3, 5}/;` → caracteres ‘a’ o ‘b’ repetidos entre 3 y 5 veces.
- **Grupo**: **( )**
  - `var er = /(hola){3}/;` → secuencia ‘hola’ que se repite 3 veces.
  - `var er = /(ab\d{2}){3}/;` → secuencia ‘ab’ seguida de dos dígitos, todo ello repetido tres veces.
- **Alternativa**: **(|)**
  - `var er = /(hola|hello){3}/;` → secuencia ‘hola’ o ‘hello’ repetida 3 veces. Ejemplo “holahellohello”.

Expresiones regulares

- Comprobar si una cadena cumple una expresión regular:
  - `er.test(cadena);`
  - Si retorna `true`, la cadena cumple el patrón.
- **Consejos de uso**:
  - Las expresiones regulares **buscan patrones**.

    ```
    var er=/hola/;
    er.test("abcahola abc"); // true
    ```
  - Habitualmente nos interesa que una **cadena completa** cumpla un patrón. Utilizaremos los **caracteres especiales ^** y **$** para delimitar el **principio y fin** del patrón.

    ```
    var er=/^hola$/;
    er.test("abcahola abc"); // false
    ```

- Las expresiones regulares se utilizan con los métodos de **RegExp test** y **exec** y con los metodos de **String match**, **replace**, **search**, and **split**

  - **exec** Un método RegExp que ejecuta una búsqueda por una coincidencia en una cadena. Devuelve un array de información.

  - **test** Un método RegExp que verifica una coincidencia en una cadena. Devuelve true o false.

- **match** Un método String que ejecuta una búsqueda por una coincidencia en una cadena. Devuelve un array de información o null si no existe coincidencia alguna.
- **search** Un método String que verifica una coincidencia en una cadena. Devuelve el índice de la coincidencia, o -1 si la búsqueda falla.
- **replace** Un método String que ejecuta una búsqueda por una coincidencia en una cadena, y reemplaza la subcadena encontrada con una subcadena de reemplazo.
- **split** Un método String que utiliza una expresión regular o una cadena fija para cortar una cadena y colocarlo en un array de subcadenas.

- Cuando quiera saber si un patrón se encuentra en una cadena, utilice los métodos **test** o **search**.

- Para obtener más información (pero ejecución más lenta) utilice los métodos **exec** o **match**.

```
var re = /(\w+)\s(\w+)/;
var str = "John Smith";
var newstr = str.replace (re, '$2, $1');
console.log (newstr);  //devuelve "Smith, John"

// patrón: posible espacio en blanco a
   continuación, punto y coma entonces posible
   espacio en blanco
var names = "Harry Trump; Fred Barney; Helen Rigby; Bill Abel"
var pattern = /\s*;\s*/;
// Romper la cadena en piezas separadas por el
   patrón y almacenar en un array
var NAMELIST = names.split (patrón);
```

Expresiones regulares

- **Consejos de uso** (continuación):
  - Hay que llevar cuidado con los **caracteres especiales** al definir las expresiones. Utilizamos el carácter de escape "\"

    `/^\w+\.\w*$/` → uno o más caracteres alfanuméricos que incluyen un "."

  - No se pueden definir **rangos numéricos**.

    `/^[12-40]$/` → "1", "2", "3", "4" ó "0". En el patrón, 2-4 define un rango de caracteres.

Ejercicios

1 - Solicita el ingreso de la cordenada de un punto. El formato a ingresar por teclado es (999,999). Los números pueden tener entre 1 y 3 dígitos.

2 - Valida el ingreso de una hora con el formato hh:mm:ss o hh:mm

3 - Valida un mail

4 - Valida el ingreso del nombre de un día de la semana y un número de 1 o 2 dígitos

5 - Validar DNI

6 - Validar un número Hexadecimal

7 - Valida un nombre de usuario en twitter, empieza por @ y puede contener letras mayusculas y minusculas, numeros, guiones y guiones bajos.

8 - Validar ISBN de 13 digitos, siempre empieza en 978 o 979.